



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Pablo Arturo Gonzalez Cuellar

N° de Cuenta: 319241013

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 21/09/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Terminar la Grúa con:

- cuerpo(prisma rectangular)
- base (pirámide cuadrangular)
- 4 llantas(4 cilindros) con teclado se pueden girar cada una de las 4 llantas por separado

Primeramente, se corrigió la jerarquía del modelo, antes, se tenía un modelo donde la base de la grúa era el objeto padre de la cabina, cuando lógicamente y basado en el árbol jerárquico se debía implementar al revés.

Para esto, se modificó el orden de renderizado. Primero se dibujó la cabina y la guardamos en su matriz de transformación y, a partir de esta matriz, se dibuja la base y los brazos articulados como sus hijos.

```
// Cabina como padre
model = glm::translate(model, glm::vec3(0.0f, 2.0f, -4.0f));
modelaux = model; // Guardamos la matriz de la cabina
model = glm::scale(model, glm::vec3(4.0f, 2.0f, 3.0f));
color = glm::vec3(1.0f, 0.0f, 0.0f); // Color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();

// Base como hijo de la cabina
model = modelaux; // Partimos de la cabina
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
modelaux2 = model; // Guardamos la matriz de la base para las llantas
model = glm::scale(model, glm::vec3(5.0f, 2.5f, 4.0f));
color = glm::vec3(0.9f, 0.6f, 0.1f); // Color naranja
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2]->RenderMesh(); // Usamos la pirámide
```

Una vez teniendo esto pasamos al principal objetivo de este primer punto del desarrollo de la práctica, el cual es el moviendo de las ruedas.

El requerimiento es añadir la capacidad de rotar cada una de las cuatro ruedas de forma independiente. Para esto, primero se modificó la clase `Window.h` para manejar 8 nuevas teclas y almacenar los ángulos de rotación.

```
GLfloat getArticulacion6() { return articulacion6; }
GLfloat getGiroLlantaDD() { return giroLlantaDD; } // Delantera Derecha
GLfloat getGiroLlantaDI() { return giroLlantaDI; } // Delantera Izquierda
GLfloat getGiroLlantaTD() { return giroLlantaTD; } // Trasera Derecha
GLfloat getGiroLlantaTI() { return giroLlantaTI; } // Trasera Izquierda

~Window();
private:
    GLFWwindow* mainWindow;
    GLint width, height;
    GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6;
    GLfloat giroLlantaDD, giroLlantaDI, giroLlantaTD, giroLlantaTI;
    bool keys[1024];
```

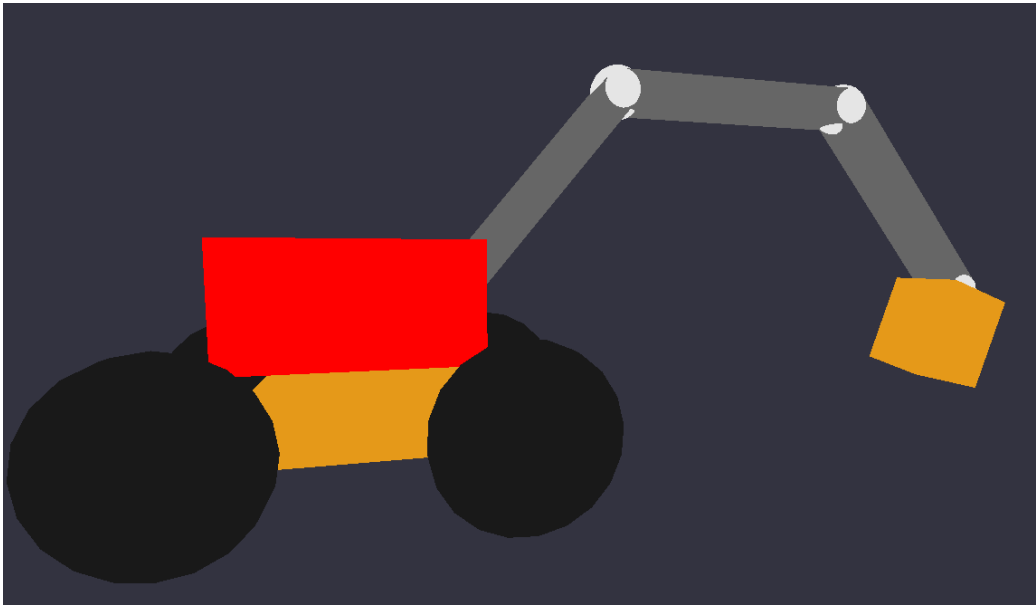
Al igual, se añadió la lógica para procesar las nuevas teclas en `Window.cpp`

```
if (key == GLFW_KEY_1) theWindow->giroLlantaDD += 5.0;
if (key == GLFW_KEY_2) theWindow->giroLlantaDD -= 5.0;
// Llanta Delantera Izquierda
if (key == GLFW_KEY_3) theWindow->giroLlantaDI += 5.0;
if (key == GLFW_KEY_4) theWindow->giroLlantaDI -= 5.0;
// Llanta Trasera Derecha
if (key == GLFW_KEY_5) theWindow->giroLlantaTD += 5.0;
if (key == GLFW_KEY_6) theWindow->giroLlantaTD -= 5.0;
// Llanta Trasera Izquierda
if (key == GLFW_KEY_7) theWindow->giroLlantaTI += 5.0;
if (key == GLFW_KEY_8) theWindow->giroLlantaTI -= 5.0;
```

Finalmente, en nuestro `main` se aplicó esta rotación a cada llanta. Toda esta transformación se realizó sobre el eje Z, que una vez “acostada”, funciona como el eje de la llanta y por lo tanto produciendo el efecto de rodamiento.

```
// Llanta delantera derecha
model = modelaux2; // Partimos de la Base
model = glm::translate(model, glm::vec3(2.5f, -1.0f, 2.2f));
// Rotación de la rueda
model = glm::rotate(model, glm::radians(mainWindow.getGiroLlantaDD()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 0.3f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMeshGeometry();
```

CAPTURAS DE EJECUCIÓN:



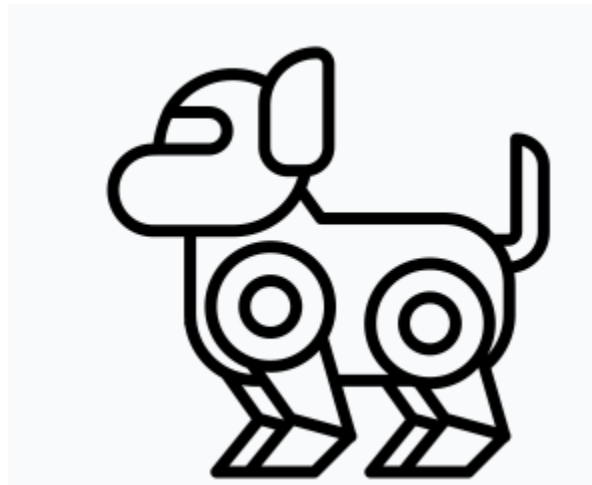
2.- Crear un animal robot 3d

-Instanciando cubos, pirámides, cilindros, conos, esferas:

-4 patas articuladas en 2 partes (con teclado se puede mover las dos articulaciones de cada pata)

-cola articulada o 2 orejas articuladas. (con teclado se puede mover la cola o cada oreja independiente

Para esta segunda parte de la práctica se eligió modelar un gato robot tomando como modelo el de la imagen, ya que su estructura se adaptaba bien a las implementaciones que ya se tenían.



Para comenzar el modelado de este robot, se comenzó por el cuerpo, ya que éste actuará como el nodo padre de toda la jerarquía del modelado. Se modeló con dos cubos, uno para el cuerpo en general y otro para conectarlo más adelante con la cabeza.

```
// Cuerpo (Nodo padre)
model = glm::translate(model, glm::vec3(0.0f, 2.5f, 0.0f));
modelaux = model;

model = glm::scale(model, glm::vec3(2.5f, 2.0f, 4.0f));
color = glm::vec3(0.8f, 0.8f, 0.8f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();

// Base para la cabeza (Hija del cuerpo principal)
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.25f, 2.2f));

model = glm::scale(model, glm::vec3(2.0f, 1.5f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();
```

Posteriormente se modeló la cabeza como una esfera, la cual se conectó como hija de la base de la cabeza que sale del cuerpo del robot.

```
// CABEZA
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 1.5f, 3.5f));

model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();

modelaux2 = model;
```

Ahora, nos encontramos con la parte más compleja de este modelado, la cual es la implementación de las patas articuladas y los controles para mover estas.

Cada pata se modeló bajo una jerarquía de 5 niveles: la articulación superior o el “hombro” del robot, el segmento superior de la pata, la articulación inferior o la “rodilla” del robot, el segmento inferior de la pata y por último el pie. Por lo tanto, hacemos la declaraciones de las variables que vamos a utilizar así como nuestros getters.

```
GLfloat getGiroLlantaTI() { return giroLlantaTI; }
GLfloat getPataDISup() { return pataDISup; }
GLfloat getPataDIInf() { return pataDIInf; }
GLfloat getPataDDSup() { return pataDDSup; }
GLfloat getPataDDInf() { return pataDDInf; }
GLfloat getPataTISup() { return pataTISup; }
GLfloat getPataTIInf() { return pataTIInf; }
GLfloat getPataTDSup() { return pataTDSup; }
GLfloat getPataTDInf() { return pataTDInf; }
GLfloat getColaUD() { return colaUD; }
GLfloat getColaLR() { return colaLR; }

~Window();
private:
GLFWwindow *mainWindow;
GLint width, height;
GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6;
GLfloat giroLlantaDD, giroLlantaDI, giroLlantaTD, giroLlantaTI;
bool keys[1024];
```

```
giroLlantaTI = 0.0f;
```

```
// Variables para el animal robot
```

```
pataDISup = 0.0f; pataDIInf = 0.0f;
```

```
pataDDSup = 0.0f; pataDDInf = 0.0f;
```

```
pataTISup = 0.0f; pataTIInf = 0.0f;
```

```
pataTDSup = 0.0f; pataTDInf = 0.0f;
```

```
colaUD = 0.0f;
```

```
colaLR = 0.0f;
```

```

if (mode == GLFW_MOD_SHIFT) // Mover patas traseras con Sh
{
    // Pata Trasera Izquierda
    if (key == GLFW_KEY_Q) theWindow->pataTISup += 5.0;
    if (key == GLFW_KEY_1) theWindow->pataTISup -= 5.0;
    if (key == GLFW_KEY_Z) theWindow->pataTIInf += 5.0;
    if (key == GLFW_KEY_X) theWindow->pataTIInf -= 5.0;

    // Pata Trasera Derecha
    if (key == GLFW_KEY_E) theWindow->pataTDSup += 5.0;
    if (key == GLFW_KEY_2) theWindow->pataTDSup -= 5.0;
    if (key == GLFW_KEY_C) theWindow->pataTDInf += 5.0;
    if (key == GLFW_KEY_V) theWindow->pataTDInf -= 5.0;
}
else // Mover patas delanteras y cola sin SHIFT
{
    // Pata Delantera Izquierda
    if (key == GLFW_KEY_Q) theWindow->pataDISup += 5.0;
    if (key == GLFW_KEY_1) theWindow->pataDISup -= 5.0;
    if (key == GLFW_KEY_Z) theWindow->pataDIInf += 5.0;
    if (key == GLFW_KEY_X) theWindow->pataDIInf -= 5.0;

    // Pata Delantera Derecha
    if (key == GLFW_KEY_E) theWindow->pataDDSup += 5.0;
    if (key == GLFW_KEY_2) theWindow->pataDDSup -= 5.0;
    if (key == GLFW_KEY_C) theWindow->pataDDInf += 5.0;
    if (key == GLFW_KEY_V) theWindow->pataDDInf -= 5.0;
}

```

Para el movimiento, se decidió hacer uso de las teclas Q,1,E,2,Z,X,C,V y `SHIFT` para alternar entre las patas delanteras y traseras, evitando tener que declarar más teclas dispersas en el teclado.

Ahora bien, para nuestro main, cada pata sigue la jerarquía planeada, donde cada componente se posiciona inmediato al anterior.

```

    ///      Patatas (Hijas del cuerpo principal)      ///

    // Pata Delantera Izquierda
    model = modelaux;
    model = glm::translate(model, glm::vec3(-1.2f, -0.5f, 1.8f));
    model = glm::rotate(model, glm::radians(mainWindow.getPataDISup()), glm::vec3(1.0f, 0.0f, 0.0f));
    modelaux2 = model;

    // Esfera del hombro
    model = glm::scale(model, glm::vec3(0.5f));
    color = glm::vec3(0.2f, 0.2f, 0.2f);
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    sp.render();

    // Parte Superior de la Pata
    model = modelaux2;
    model = glm::translate(model, glm::vec3(0.0f, -0.9f, 0.0f));
    model = glm::scale(model, glm::vec3(0.4f, 1.8f, 0.4f));
    color = glm::vec3(0.8f, 0.8f, 0.8f);
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    meshList[1]->RenderMeshGeometry();

    // Rodilla
    model = modelaux2;
    model = glm::translate(model, glm::vec3(0.0f, -1.8f, 0.0f));

```

Después pasamos al modelado de la cola articulada, la cual se modeló con 3 cilindros, donde la base de la cola tiene movimiento controlado por las teclas F,T,G y H, mientras que los segmentos posteriores tienen una rotación fija para darle una curvatura a la cola.

```

    // Cola
    if (key == GLFW_KEY_T) theWindow->colaUD += 5.0;
    if (key == GLFW_KEY_G) theWindow->colaUD -= 5.0;
    if (key == GLFW_KEY_F) theWindow->colaLR += 5.0;
    if (key == GLFW_KEY_H) theWindow->colaLR -= 5.0;
}

```

```

bool mouseFirstMoved;
void createCallbacks();
static void ManejaTeclado(GLFWwindow* window, int key, int cod
static void ManejaMouse(GLFWwindow* window, double xPos, doubl
GLfloat pataDISup, pataDIInf, pataDDSup, pataDDInf, pataTISup,
GLfloat colaUD, colaLR;
};

```

Implementación del modelado:


```

// Cola (Se descomponerá en 3 segmentos)
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.8f, -2.2f));

model = glm::rotate(model, glm::radians(mainWindow.getColaUD()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getColaLR()), glm::vec3(0.0f, 1.0f, 0.0f));

// Segmento 1
modelaux2 = model;
model = glm::translate(model, glm::vec3(0.0f, 0.4f, 0.0f));
model = glm::scale(model, glm::vec3(0.2f, 0.8f, 0.2f));

color = glm::vec3(0.2f, 0.2f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMeshGeometry();

// Segmento 2
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, 0.8f, 0.0f));
model = glm::rotate(model, glm::radians(15.0f), glm::vec3(1.0f, 0.0f, 0.0f));
modelaux2 = model;

```

Finalmente, se añadieron detalles menores como las orejas, las cuales se modelaron como 2 pirámides, lo ojos, con dos esferas, y un suelo para completar el escenario

```

//      OREJAS      //

// Oreja Derecha
model = modelaux2;
model = glm::translate(model, glm::vec3(0.5f, 0.7f, 0.0f));
model = glm::rotate(model, glm::radians(-15.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.6f, 1.0f, 0.6f));

color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2]->RenderMesh();

// Oreja Izquierda
model = modelaux2;
model = glm::translate(model, glm::vec3(-0.5f, 0.7f, 0.0f));
model = glm::rotate(model, glm::radians(15.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.6f, 1.0f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2]->RenderMesh();

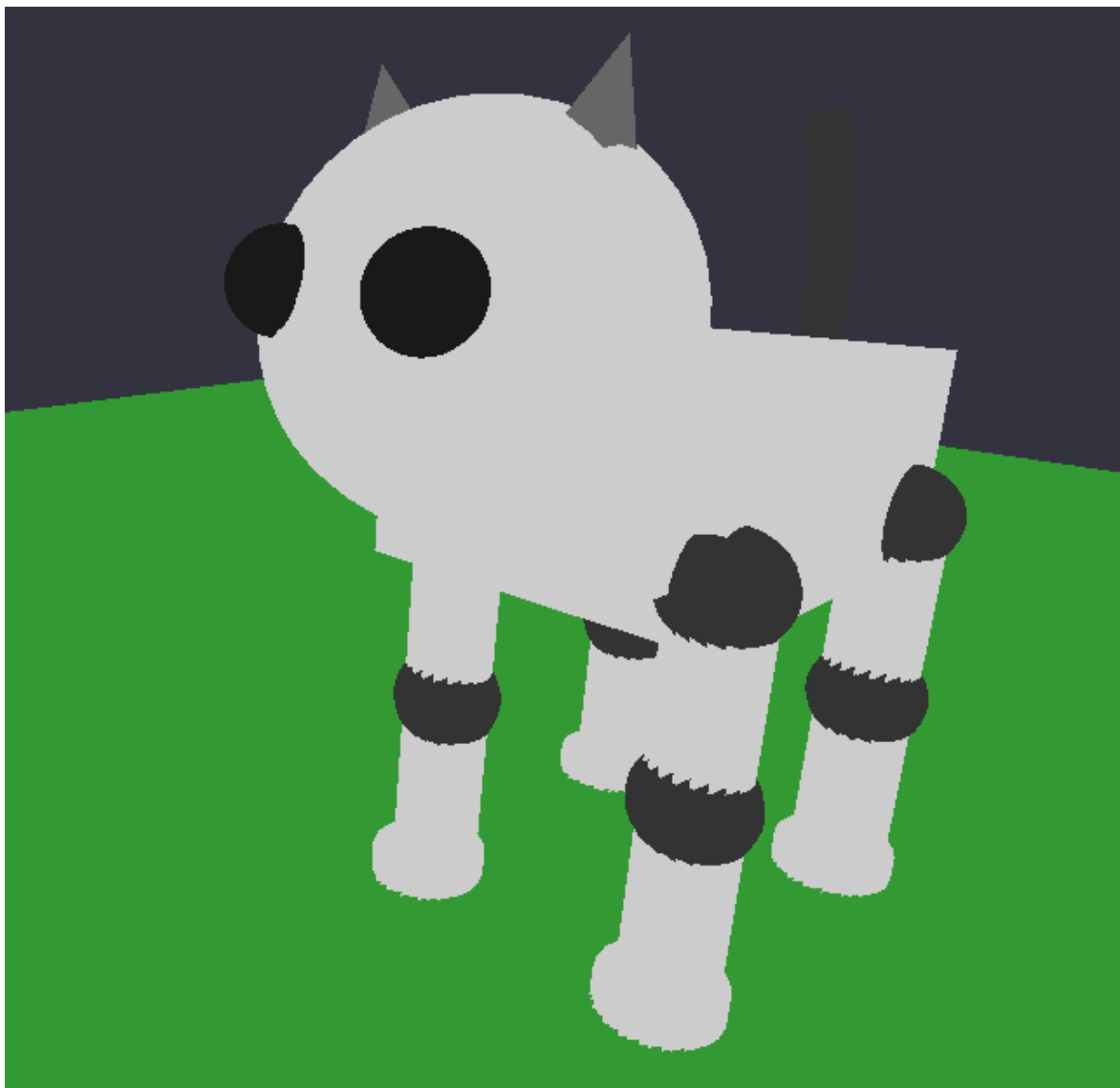
// Ojos (Hijos de la cabeza)
color = glm::vec3(0.1f, 0.1f, 0.1f); // Color negro para los ojos
glUniform3fv(uniformColor, 1, glm::value_ptr(color));

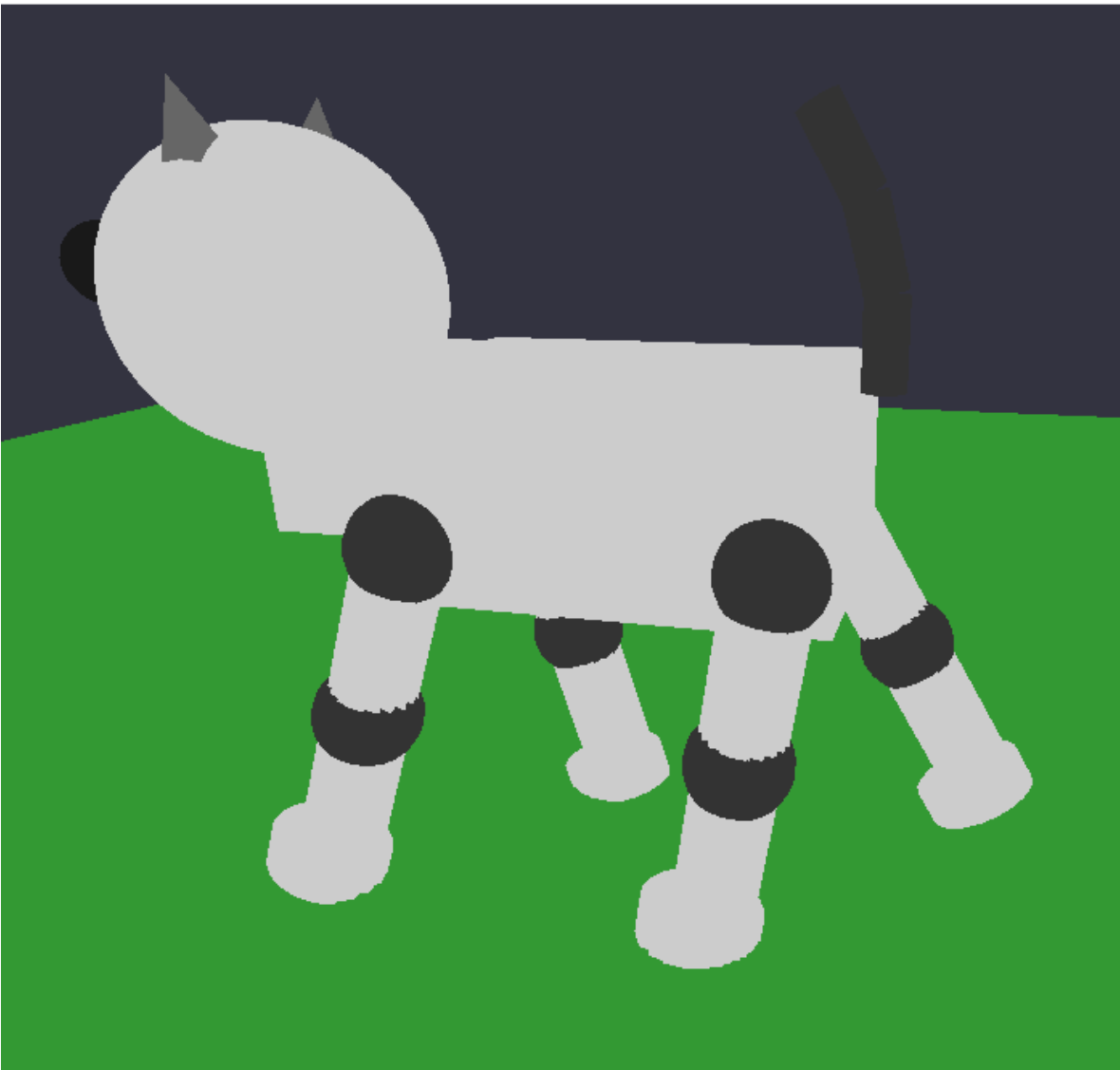
// Ojo Derecho
model = modelaux2; // Partimos del centro de la cabeza
model = glm::translate(model, glm::vec3(0.4f, 0.2f, 0.85f)); // Hacemos traslacion
model = glm::scale(model, glm::vec3(0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();

// Ojo Izquierdo
model = modelaux2; // Partimos de nuevo del centro de la cabeza
model = glm::translate(model, glm::vec3(-0.4f, 0.2f, 0.85f)); // Movemos a la izqui

```

CAPTURAS DE EJECUCIÓN:







PROBLEMAS SURGIDOS:

El diseño e implementación de las articulaciones para las patas del robot fue un tema difícil, en el cual surgieron dos principales problemas, primero, se tuvo problemas al aplicar un escalado antes de que todas las rotaciones de una sola pata estuvieran completas. Aquí, los segmentos de las patas se estiraban y deformaban de forma extraña al ser rotados. Al ir probando con diversas soluciones, me di cuenta que la causa era que el escalado no solo estaba afectando al tamaño del objeto, sino que también a la posición relativa donde se encontraba si no se aplicaba en el orden correcto. Por lo tanto, la solución fue aplicar el escalado después de haberlo posicionado y rotado correctamente.

Otro problema, y algo relacionado con el anterior, fue que al tener múltiples segmentos para las patas, era muy importante saber cuando guardar y restaurar el estado de la matriz de modelo. En varias ocasiones, al modelar la rodilla, olvidé restaurar la matriz del hombro, lo que provocaba que toda la pata se desprendiera del cuerpo o se moviera a una posición no deseada.

CONCLUSIÓN

Esta práctica fue un ejercicio muy laborioso pero importante, ya que se reforzó la importancia del modelado jerárquico. La primera parte, con la grúa, me enseñó la importancia de tener una estructura ordenada, entender que la cabina debía ser el nodo padre de las demás piezas para que todos los demás componentes que dependieran de ella se modelaran de la mejor manera. La segunda parte, fue donde de verdad me puse a prueba. El ensamblado pieza por pieza de la cabeza, el cuerpo, orejas y cada articulación de las patas, fue un desafío arduo, ya que, el hacer que todas las partes móviles obedecieran al teclado y que trabajaran en conjunto trabajaba principios de modelo jerárquico que se tuvieron que reforzar.